计算机学院实验报告

实验题目： 三种平面三角形点定位算法		学号：202200130077
日期：2024.3.16	班级：智能	姓名：薛润泽
Email：		

实验目的：

实验要求：

1， 第一种算法为射线求交法（根据交点个数判定）；
2， 第二种算法任选；
3， 有余力的同学欢迎实现第三种算法；
4， 在实验 0 的平台上开发；
5， 要求写实验报告，具体提交方式助教发布；

实验环境介绍：

Ubuntu22.04

解决问题的主要思路：

本实验的主要思路如下：

1. 学习使用射线交点法，向量叉乘判断方向法和面积法来判断点与三角形的位置关系。
2. 学习使用 OpenCV 进行点与线段的可视化。

其中，射线交点法思路：做过判断点的一条射线，我在此将其做成一条平行于任意三角形边的射线。特殊判断该点是否在三角形边上，是否在三角形三个顶点上，该射线是否与三角形边重合后，计算该射线与三角形的交点数目。如果交点数目为偶数个（0 或 2 个），说明该点在三角形外侧，如果交点个数为奇数个（1 个），说明该点在三角形内侧。

向量叉乘判断方向法思路：将该点与三角形三顶点相连，连同三角形三边形成六个向量。点与顶点相连的向量以及该顶点与下一个顶点向量的向量做叉乘，可以得到一个新向量。将每个顶点如此操作后共得到 3 个向量。如果该点在三角形的内侧，则该 3 个向量的方向相同；否则该点在三角形的外侧。

面积法思路：如果该点在三角形内侧，则该点与三角形三个点中两两点形成的三个三角形面积之和等于原三角形面积；如果点在三角形内侧，则三个三角形面积之和大于原三角形面积。

实验步骤与实验结果：

1. 射线交点法

射线交点法的关键在于怎么求射线与线段之间的交点。我们使用一个较长的线段来表示“射线”，则可以转化为求线段与线段之间的交点了。怎么求线段与线段之间的交点？可以使用向量叉乘的方法，设第一条线段 AB， 第二条线段 CD。如果向量 AB×BC ，BC×CD，CD×AD 以及 AB×BD，BD×CD，CD×AB 两组向量解中存在一组向量解的方向相同，则说明两线段不相交。否则相交。

下面是代码实现：

```
#include<cmath>
#include<eigen3/Eigen/Core>
#include<eigen3/Eigen/Dense>
#include<iostream>
#include <opencv2/core/mat.hpp>
#include <opencv2/opencv.hpp>

double VectorCross(double x1, double y1, double x2, double
y2){
    return x1*y2 - x2*y1;
}
int checkpoints(double x1, double y1, double x2, double
y2,double x3, double y3, double x4, double y4){

    double xp1_p2 = x2 - x1, xp2_p3 = x3 - x2, xp3_p4 = x4 -
x3, xp4_p1 = x1 - x4, xp2_p4 = x4 - x2, xp4_p3 = x3 - x4,
xp3_p1 = x1 - x3;
    double yp1_p2 = y2 - y1, yp2_p3 = y3 - y2, yp3_p4 = y4 -
y3, yp4_p1 = y1 - y4, yp2_p4 = y4 - y2, yp4_p3 = y3 - y4,
yp3_p1 = y1 - y3;
    double ans1 = VectorCross(xp1_p2, yp1_p2, xp2_p3, yp2_p3)
* VectorCross(xp3_p4, yp3_p4, xp4_p1, yp4_p1);
    double ans2 = VectorCross(xp1_p2, yp1_p2, xp2_p4, yp2_p4)
* VectorCross(xp4_p3, yp4_p3, xp3_p1, yp3_p1);
    if(ans1 == 0 || ans2 == 0)return 1;// same line
    else if(ans1 > 0 || ans2 > 0) return 0;// NO cross point
xian duan
    else return 1;//Have corss point
}

int Solve(double x1, double y1, double x2,double y2,double
x3,double y3, double test1x, double test1y){
```

```
    // 0 outside, 1 inside , 2 onpoints , 3 onlines
    //test1x, test1y
    //special start
    if((test1x == x1 && test1y == y1) || (test1x == x2 &&
test1y == y2) || (test1x == x3 && test1y == y3))return 1;
    if(VectorCross(x1-x3, y1-y3, x1-test1x, y1 -test1y) == 0){
        // inner or out
        if((test1x - x1) * (test1x - x3) + (test1y -
y1)*(test1y - y3) > 0) return 0;
        else return 1;
    }
    if(VectorCross(x1-x2, y1-y2, x1- test1x, y1-test1y) == 0){
        if((test1x - x1) * (test1x - x2) + (test1y -
y1)*(test1y - y2) > 0) return 0;
        else return 1;
    }
    if(VectorCross(x2-x3, y2-y3, x2 - test1x, y2-test1y) ==
0){
        if((test1x - x2) * (test1x - x3) + (test1y -
y2)*(test1y - y3) > 0) return 0;
        else return 1;
    }
    //special end
    double minor_x1_x2 = x2 - x1;
    double minor_y1_y2 = y2 - y1;
    double virtualx = test1x + minor_x1_x2;
    double virtualy = test1y + minor_y1_y2;
    virtualx = virtualx * 1000000;
    virtualy = virtualy * 1000000;
    int sym1 = checkpoints(test1x, test1y, virtualx, virtualy,
x2, y2, x3 ,y3);
    int sym2 = checkpoints(test1x, test1y, virtualx, virtualy,
x3, y3, x1, y1);
    int sym = sym1+sym2;
    if(sym == 2) sym =0;
    return sym;
}
int main(){
    double x1 , y1, x2, y2, x3, y3, testx, testy;
    cv::Mat img = cv::Mat::zeros(512,512,CV_8UC3);
    cv::imshow("Image",img);
    cv::waitKey(5000);
    scanf("%lf%lf",&x1,&y1);
    cv::Point center1(x1,y1);
```

```
    cv::circle(img, center1, 5,cv::Scalar(0,0,255),
cv::FILLED);
    cv::imshow("Image",img);
    cv::waitKey(5000);
    scanf("%lf%lf",&x2, &y2);
    cv::Point center2(x2,y2);
    cv::circle(img, center2, 5,cv::Scalar(0,0,255),
cv::FILLED);
    cv::line(img, cv::Point(x1,y1), cv::Point(x2,y2),
cv::Scalar(255,0,0),2);
    cv::imshow("Image",img);
    cv::waitKey(5000);
    scanf("%lf%lf",&x3,&y3);
    cv::Point center3(x3,y3);
    cv::circle(img, center3, 5,cv::Scalar(0,0,255),
cv::FILLED);
    cv::line(img, cv::Point(x1,y1), cv::Point(x3,y3),
cv::Scalar(255,0,0),2);
    cv::line(img, cv::Point(x2,y2), cv::Point(x3,y3),
cv::Scalar(255,0,0),2);
    cv::imshow("Image",img);
    cv::waitKey(5000);
    scanf("%lf%lf",&testx,&testy);
    cv::Point center4(testx, testy);
    cv::circle(img, center4, 5,cv::Scalar(0,0,255),
cv::FILLED);
    cv::imshow("Image",img);

    std::string text1 = "Inner";
    std::string text2 = "Outside";
    cv::Point org(50, 256);
    int fontFace = cv::FONT_HERSHEY_SIMPLEX;
    double fontScale = 1;
    cv::Scalar color(0,255,0);
    int thickness = 2;
    if(Solve(x1,y1,x2,y2,x3,y3,testx,testy) == 1)
cv::putText(img, text1, org, fontFace, fontScale, color,
thickness);
    else cv::putText(img, text2, org, fontFace, fontScale,
color, thickness);
    cv::imshow("Image", img);

    cv::waitKey(0);
}
```

实验效果：三种方法得到的效果相同，在方法 3 处展示

2、向量叉乘判断方向法

向量叉乘判断方向法要计算六个向量，以及三组向量叉乘。如果三组向量叉乘的方向相同，说明点在三条线段的同一侧，所以点在三角形内。

下面是代码实现：

```
#include<cmath>
#include<eigen3/Eigen/Core>
#include<eigen3/Eigen/Dense>
#include<iostream>
#include <opencv2/core/mat.hpp>
#include <opencv2/opencv.hpp>

double VectorCross(double x1, double y1, double x2, double y2){
    return x1*y2 - x2*y1;
}
int checkpoints(double x1, double y1, double x2, double
y2,double x3, double y3, double testx, double testy){
    double xp1_p2 = x2 - x1, xpt_p1 = x1 - testx, xp2_p3 = x3 -
x2, xpt_p2 = x3 - testx, xp3_p1 = x1 - x3, xpt_p3 = x3 - testx;
    double yp1_p2 = y2 - y1, ypt_p1 = y1 - testy, yp2_p3 = y3 -
y2, ypt_p2 = y3 - testy, yp3_p1 = y1 - y3, ypt_p3 = y3 - testy;
    double ans1, ans2, ans3;
    ans1 = VectorCross(xpt_p1, ypt_p1, xp1_p2, yp1_p2);
    ans2 = VectorCross(xpt_p2, ypt_p2, xp2_p3, yp2_p3);
    ans3 = VectorCross(xpt_p3, ypt_p3, xp3_p1, yp3_p1);
    if((ans1 < 0 && ans2 < 0 && ans3 < 0)||(ans1 > 0 && ans2 > 0
&& ans3 > 0))return 1;//in
    else return 0;//not in inner
}

int main(){
double x1 , y1, x2, y2, x3, y3, testx, testy;
    cv::Mat img = cv::Mat::zeros(512,512,CV_8UC3);
    cv::imshow("Image",img);
    cv::waitKey(5000);
    scanf("%lf%lf",&x1,&y1);
    cv::Point center1(x1,y1);
    cv::circle(img, center1, 5,cv::Scalar(0,0,255), cv::FILLED);
    cv::imshow("Image",img);
    cv::waitKey(5000);
```

```
    scanf("%lf%lf",&x2, &y2);
    cv::Point center2(x2,y2);
    cv::circle(img, center2, 5,cv::Scalar(0,0,255), cv::FILLED);
    cv::line(img, cv::Point(x1,y1), cv::Point(x2,y2),
cv::Scalar(255,0,0),2);
    cv::imshow("Image",img);
    cv::waitKey(5000);
    scanf("%lf%lf",&x3,&y3);
    cv::Point center3(x3,y3);
    cv::circle(img, center3, 5,cv::Scalar(0,0,255), cv::FILLED);
    cv::line(img, cv::Point(x1,y1), cv::Point(x3,y3),
cv::Scalar(255,0,0),2);
    cv::line(img, cv::Point(x2,y2), cv::Point(x3,y3),
cv::Scalar(255,0,0),2);
    cv::imshow("Image",img);
    cv::waitKey(5000);
    scanf("%lf%lf",&testx,&testy);
    cv::Point center4(testx, testy);
    cv::circle(img, center4, 5,cv::Scalar(0,0,255), cv::FILLED);
    cv::imshow("Image",img);

    std::string text1 = "Inner";
    std::string text2 = "Outside";
    cv::Point org(50, 256);
    int fontFace = cv::FONT_HERSHEY_SIMPLEX;
    double fontScale = 1;
    cv::Scalar color(0,255,0);
    int thickness = 2;
    if(checkpoints(x1,y1,x2,y2,x3,y3,testx,testy) == 1)
cv::putText(img, text1, org, fontFace, fontScale, color,
thickness);
    else cv::putText(img, text2, org, fontFace, fontScale,
color, thickness);
    cv::imshow("Image", img);

    cv::waitKey(0);
}
```

实验效果：在方法 3 处展示

3、面积法

面积法比较直观，如果将点在三角形内侧，则点与三顶点形成的三角形面积等于原三角形。如果点在三角形外侧，则点与三顶点形成的三角形面积大于原三角形（因为形成的图形把原三角形完全覆盖，且多出一块面积）

下面是代码示例：

```
#include<cmath>
#include<eigen3/Eigen/Core>
#include<eigen3/Eigen/Dense>
#include<iostream>
#include <opencv2/core/mat.hpp>
#include <opencv2/opencv.hpp>

double VectorCross(double x1, double y1, double x2, double y2){
    return x1*y2 - x2*y1;
}
int checkpoints(double x1, double y1, double x2, double
y2,double x3, double y3, double testx, double testy){
    double xp1_p2 = x2 - x1, xp2_p3 = x3 - x2, xpt_p1 = x1-
testx, xpt_p2 = x2 - testx, xpt_p3 = x3 - testx;
    double yp1_p2 = y2 - y1, yp2_p3 = y3 - y2, ypt_p1 = y1-
testy, ypt_p2 = y2 -testy, ypt_p3 = y3 -testy;
    double ans1, ans2, ans3;
    double Sorigin = abs(VectorCross(xp1_p2, yp1_p2, xp2_p3,
yp2_p3));
    ans1 = abs(VectorCross(xpt_p1,ypt_p1,xpt_p2,ypt_p2));
    ans2 = abs(VectorCross(xpt_p2, ypt_p2, xpt_p3, ypt_p3));
    ans3 = abs(VectorCross(xpt_p3, ypt_p3, xpt_p1, ypt_p1));
    if(abs(ans1 + ans2 + ans3 - Sorigin) <= 1) return 1;
    else return 0;//not in inner
}

int main(){
double x1 , y1, x2, y2, x3, y3, testx, testy;
    cv::Mat img = cv::Mat::zeros(512,512,CV_8UC3);
    cv::imshow("Image",img);
    cv::waitKey(5000);
    scanf("%lf%lf",&x1,&y1);
    cv::Point center1(x1,y1);
    cv::circle(img, center1, 5,cv::Scalar(0,0,255), cv::FILLED);
    cv::imshow("Image",img);
    cv::waitKey(5000);
    scanf("%lf%lf",&x2, &y2);
    cv::Point center2(x2,y2);
    cv::circle(img, center2, 5,cv::Scalar(0,0,255), cv::FILLED);
    cv::line(img, cv::Point(x1,y1), cv::Point(x2,y2),
cv::Scalar(255,0,0),2);
```

```
    cv::imshow("Image",img);
    cv::waitKey(5000);
    scanf("%lf%lf",&x3,&y3);
    cv::Point center3(x3,y3);
    cv::circle(img, center3, 5,cv::Scalar(0,0,255), cv::FILLED);
    cv::line(img, cv::Point(x1,y1), cv::Point(x3,y3),
cv::Scalar(255,0,0),2);
    cv::line(img, cv::Point(x2,y2), cv::Point(x3,y3),
cv::Scalar(255,0,0),2);
    cv::imshow("Image",img);
    cv::waitKey(5000);
    scanf("%lf%lf",&testx,&testy);
    cv::Point center4(testx, testy);
    cv::circle(img, center4, 5,cv::Scalar(0,0,255), cv::FILLED);
    cv::imshow("Image",img);

    std::string text1 = "Inner";
    std::string text2 = "Outside";
    cv::Point org(50, 256);
    int fontFace = cv::FONT_HERSHEY_SIMPLEX;
    double fontScale = 1;
    cv::Scalar color(0,255,0);
    int thickness = 2;
    if(checkpoints(x1,y1,x2,y2,x3,y3,testx,testy) == 1)
cv::putText(img, text1, org, fontFace, fontScale, color,
thickness);
    else cv::putText(img, text2, org, fontFace, fontScale,
color, thickness);
    cv::imshow("Image", img);

    cv::waitKey(0);



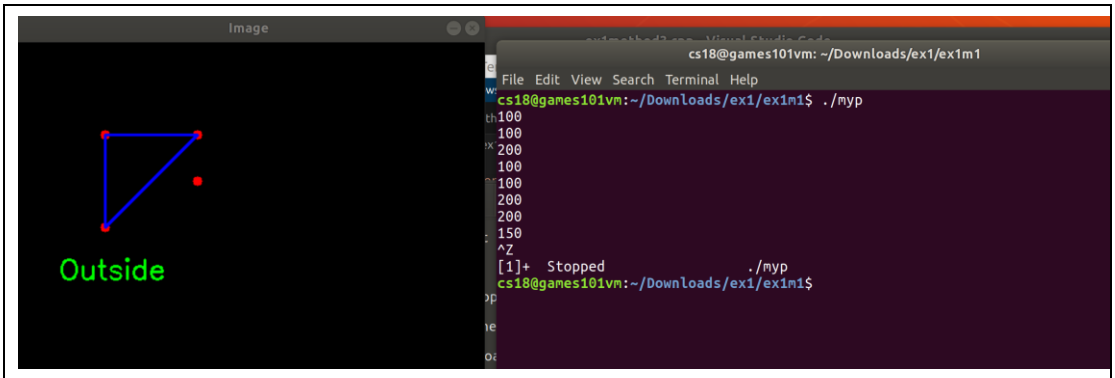


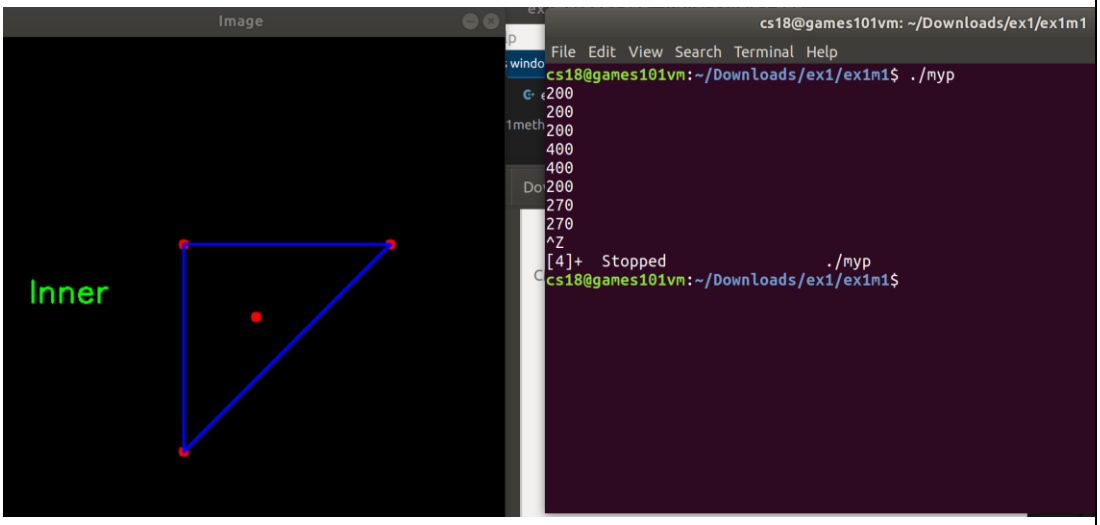




}
```

实验效果：





实验中存在的问题及解决：

问题 1：怎么判断射线与线段是否相交？

答 1：射线交点法的关键在于怎么求射线与线段之间的交点。我们使用一个较长的线段来表示“射线”，则可以转化为求线段与线段之间的交点了。怎么求线段与线段之间的交点？可以使用向量叉乘的方法，设第一条线段 AB， 第二条线段 CD。如果向量 AB×BC ，BC×CD，CD×AD 以及 AB×BD，BD×CD，CD×AB 两组向量解中存在一组向量解的方向相同，则说明两线段不相交。否则相交。

问题 2：方法 2 向量叉乘判断方向法的合理性？
因为叉乘满足右手法则，所以如果已知三个叉乘向量方向且叉乘左侧和右侧向量是确定的情况下，就可以知道点在三角形边的左右侧位置。所以如果三个叉乘向量方向相同，则可以说明点在三角形三边的同一个方向。因为三边闭合形成三角形，那么此时满足在三角形三边同一个方向（左侧或者右侧）的只有在三角形内部的点。所以是合理的。